

How to build full CRUD operations using SQLite in an ASP.NET Core Web API.

1. Overview: Why SQLite with ASP.NET Core?

SQLite is a **file-based** lightweight relational database, perfect for:

- Learning and prototyping
- Small services/microservices
- Local development
- Applications without heavy concurrency requirements

ASP.NET Core works seamlessly with SQLite through **Entity Framework Core**.

2. Step-by-Step Implementation

Step 1 — Create the ASP.NET Core Web API Project

Run:

```
dotnet new webapi -n CrudWithSQLiteDemo
```

```
cd CrudWithSQLiteDemo
```

Step 2 — Install EF Core + SQLite Packages

```
dotnet add package Microsoft.EntityFrameworkCore
```

```
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
```

```
dotnet add package Microsoft.EntityFrameworkCore.Tools
```

These packages allow the API to talk to SQLite using EF Core.

Step 3 — Create the Entity (Model)

Create folder **Models/Product.cs**

```
namespace CrudWithSQLiteDemo.Models
```

```
{  
    public class Product  
    {  
        public int Id { get; set; }           // PK  
        public string Name { get; set; } = string.Empty;  
        public decimal Price { get; set; }  
    }  
}
```

Step 4 — Create the EF Core DbContext

Create folder **Data/AppDbContext.cs**

using Microsoft.EntityFrameworkCore;

using CrudWithSQLiteDemo.Models;

namespace CrudWithSQLiteDemo.Data

```
{  
    public class AppDbContext : DbContext  
    {  
        public AppDbContext(DbContextOptions<AppDbContext> options)  
            : base(options) { }  
  
        public DbSet<Product> Products => Set<Product>();  
    }  
}
```

Step 5 — Configure SQLite in Program.cs

Open **Program.cs** and add:

```
using CrudWithSQLiteDemo.Data;
```

```
using Microsoft.EntityFrameworkCore;
```

```
var builder = WebApplication.CreateBuilder(args);
```

```
// Configure SQLite
```

```
builder.Services.AddDbContext<AppDbContext>(options =>
```

```
    options.UseSqlite(builder.Configuration.GetConnectionString("DefaultConnection"));
```

```
builder.Services.AddControllers();
```

```
builder.Services.AddEndpointsApiExplorer();
```

```
builder.Services.AddSwaggerGen();
```

```
var app = builder.Build();
```

```
if (app.Environment.IsDevelopment())
```

```
{
```

```
    app.UseSwagger();
```

```
    app.UseSwaggerUI();
```

```
}
```

```
app.UseAuthorization();
```

```
app.MapControllers();
```

```
app.Run();
```

Add connection string in **appsettings.json**:

```
{  
  "ConnectionStrings": {  
    "DefaultConnection": "Data Source=products.db"  
  }  
}
```

Step 6 — Add EF Core Migrations and Create SQLite DB

Run:

```
dotnet ef migrations add InitialCreate
```

```
dotnet ef database update
```

This creates:

- Migrations folder
 - SQLite file **products.db**
-

Step 7 — Create CRUD API Controller

Create **Controllers/ProductsController.cs**

```
using CrudWithSQLiteDemo.Data;  
using CrudWithSQLiteDemo.Models;  
using Microsoft.AspNetCore.Mvc;  
using Microsoft.EntityFrameworkCore;  
  
namespace CrudWithSQLiteDemo.Controllers  
{  
    [Route("api/[controller]")]
```

[ApiController]

public class ProductsController : ControllerBase

{

private readonly AppDbContext _context;

public ProductsController(AppDbContext context)

{

_context = context;

}

// GET api/products

[HttpGet]

public async Task<ActionResult<IEnumerable<Product>>> GetAll()

{

return Ok(await _context.Products.ToListAsync());

}

// GET api/products/5

[HttpGet("{id:int}")]

public async Task<ActionResult<Product>> GetById(int id)

{

var product = await _context.Products.FindAsync(id);

if (product == null) return NotFound();

return Ok(product);

```
}
```

```
// POST api/products
```

```
[HttpPost]
```

```
public async Task<ActionResult<Product>> Create(Product product)
```

```
{
```

```
    _context.Products.Add(product);
```

```
    await _context.SaveChangesAsync();
```

```
    return CreatedAtAction(nameof(GetById), new { id = product.Id }, product);
```

```
}
```

```
// PUT api/products/5
```

```
[HttpPut("{id:int}")]
```

```
public async Task<ActionResult> Update(int id, Product product)
```

```
{
```

```
    if (id != product.Id) return BadRequest("ID mismatch");
```

```
    _context.Entry(product).State = EntityState.Modified;
```

```
    await _context.SaveChangesAsync();
```

```
    return NoContent();
```

```
}
```

```
// DELETE api/products/5
```

```
[HttpDelete("{id:int}")]
```

```
public async Task<IActionResult> Delete(int id)
{
    var product = await _context.Products.FindAsync(id);
    if (product == null) return NotFound();

    _context.Products.Remove(product);
    await _context.SaveChangesAsync();

    return NoContent();
}
}
```

8. CRUD Operations Summary

1. CREATE (POST)

Endpoint:
POST /api/products

Sample JSON:

```
{
  "name": "Laptop",
  "price": 55000
}
```

2. READ (GET)

1. Get all
GET /api/products

2. Get by ID

GET /api/products/3

3. UPDATE (PUT)

PUT /api/products/3

```
{  
  "id": 3,  
  "name": "Updated Laptop",  
  "price": 60000  
}
```

4. DELETE (DELETE)

DELETE /api/products/3

9. SQLite Database File Location

SQLite stores the DB in a single file:

CrudWithSQLiteDemo/products.db

You can inspect it using:

- SQLiteStudio
 - DB Browser for SQLite
-

10. Full Folder Structure

CrudWithSQLiteDemo

| Program.cs

| appsettings.json

|

|—Controllers

| ProductsController.cs

|

|—Data

| AppDbContext.cs

|

|—Models

 Product.cs
